

Infografía

Módulo 5 (cont.) — Física interactiva

UPB · Gestión I/2026

De dónde venimos

La clase pasada armamos cuatro ejemplos:

1. `01_hello_pymunk.py` — física headless.
2. `02_falling_box.py` — body ↔ sprite.
3. `03_spring.py` — joints (resorte).
4. `04_car.py` — composición (chasis + ruedas + motor).

Hoy: **tres cosas más** que convierten esos ejemplos en juegos reales.

corrección del auto • agarrar con el mouse • callbacks de colisión

Plan de la clase

1. **El bug del auto** — anchors locales vs. del mundo. Una línea cambia todo.
2. `05_mouse_drag.py` — bodies kinemáticos + joints dinámicos para "agarrar" cosas con el cursor.
3. `06_collision_callback.py` — pymunk te avisa cuando dos shapes chocan. Pachinko.

Si la clase pasada el insight era "pymunk hace la física, arcade dibuja", el de hoy es "ahora podemos hablarle a pymunk en vivo: input del usuario y eventos de colisión".

1. La corrección del auto

anchors locales, no del mundo

El síntoma

Al correr `04_car.py` tal como estaba la clase pasada: **las ruedas no se quedaban fijas al chasis.**

El auto temblaba, las ruedas salían disparadas, todo se desarmaba al primer `step()`.

No era un problema de fricción, masa ni gravedad. Era un bug en la **convención de coordenadas** del joint.

El bug

```
# coordenadas del mundo (chasis en x=200, y=100):  
f_joint = pymunk.PinJoint(  
    chasis_body, f_wheel_body,  
    (x + WHEEL_DX, y + WHEEL_DY),    # <- (250, 65) en el mundo  
    (0, 0),  
)
```

Parece razonable: "el pin va donde está la rueda". Pero pymunk lee ese primer anchor como **local al chasis**.

Para pymunk, ese (250, 65) significa: "el pin sale 250px a la derecha y 65px arriba **del centro del chasis**". Un punto que está fuera de la pantalla. El constraint arrastra la rueda hacia ese punto fantasma.

El fix

```
# coordenadas LOCALES al chasis:  
f_joint = pymunk.PinJoint(  
    chassis_body, f_wheel_body,  
    (WHEEL_DX, WHEEL_DY),    # (50, -35) respecto al centro del chasis  
    (0, 0),                  # centro de la rueda  
)
```

Cambian dos números. El auto se queda armado.

El anchor de la rueda ya estaba bien: `(0, 0)` es el centro del body de la rueda, que es donde queremos el pin.

La regla en pymunk

Casi siempre: locales

```
PinJoint(a, b, anchor_a, anchor_b)
DampedSpring(a, b, anchor_a, anchor_b, ...)
PivotJoint(a, b, anchor_a, anchor_b)
```

Anchor **relativos al centro** de cada body.

Excepción: el atajo de PivotJoint

```
# 3 args en vez de 4:
PivotJoint(a, b, world_pivot)
```

Pymunk convierte el punto del mundo a local internamente. Cómodo para "anclar acá, donde están ahora".

Si el constructor toma DOS anchors → locales. Si toma UN punto → mundo.

Por qué importa esta distinción ahora

En el próximo ejemplo vamos a **crear joints en runtime** (al hacer click) y vamos a usar las dos formas:

- `body.world_to_local((x, y))` para convertir el click en un anchor local del body que agarramos.
- O el atajo de 3 args de `PivotJoint`, que hace lo mismo por nosotros.

Saber distinguir local ↔ mundo deja de ser teórico cuando el usuario hace click en cualquier lado de la pantalla.

2. Agarrar con el mouse

05_mouse_drag.py

Lo que queremos

- Caja cerrada con cajas y pelotas dinámicas adentro.
- **Click + drag** sobre cualquiera → la arrastramos con el mouse.
- Soltar → vuelve a caer libremente.

Para esto vamos a necesitar **dos cosas nuevas**:

1. Un body que **representa al mouse** dentro de la física.
2. Crear/destruir un joint en respuesta a clicks (no en `__init__`).

Concepto nuevo 1: body KINEMATIC

Ya conocíamos dos tipos:

Tipo	Comportamiento
DYNAMIC (default)	gravedad lo afecta, colisiones lo empujan
STATIC	nunca se mueve. piso, paredes
KINEMATIC	se mueve solo si vos lo movés a mano. no le afecta gravedad ni colisiones

KINEMATIC = "lo controlo yo, pero existe en la física". Perfecto para el cursor, plataformas móviles, jefes con patrón de movimiento.

El cursor como body

```
self.cursor_body = pymunk.Body(body_type=pymunk.Body.KINEMATIC)
self.cursor_body.position = (WIDTH // 2, HEIGHT // 2)

def on_mouse_motion(self, x, y, dx, dy):
    self.cursor_body.position = (x, y)
```

No tiene shape: no necesita chocar con nada. Es solo "un punto fantasma" en el espacio físico para poder enganchar joints contra él.

Si fuera DYNAMIC con masa, la gravedad lo tiraría hacia abajo y nuestro `position = (x, y)` pelearía contra el motor. Si fuera STATIC, pymunk asume que **nunca** se mueve y optimiza colisiones asumiéndolo.

Concepto nuevo 2: joints dinámicos

Hasta ahora los joints se creaban en `__init__` y vivían para siempre. Pero los joints también son objetos: se pueden agregar **y quitar** del space en cualquier momento.

```
joint = pymunk.PivotJoint(a, b, ...)
space.add(joint)           # entra a vigilar la restriccion
# ... más tarde ...
space.remove(joint)        # se olvida de la restriccion
```

"Agarrar" = crear joint en `mouse_press`. **"Soltar"** = `remove joint` en `mouse_release`.

El click: encontrar qué hay debajo

```
def on_mouse_press(self, x, y, button, modifiers):
    if button != arcade.MOUSE_BUTTON_LEFT:
        return
    hit = self.space.point_query_nearest(
        (x, y), 0, pymunk.ShapeFilter(),
    )
    if hit is None or hit.shape.body.body_type != pymunk.Body.DYNAMIC:
        return
    # ... crear joint ...
```

- `point_query_nearest` busca la shape más cercana al punto (distancia `0` = "tiene que estar **sobre** el punto").
- Filtramos a solo DYNAMIC: no queremos agarrar el piso.

El click: crear el PivotJoint

```
body = hit.shape.body
local_anchor = body.world_to_local((x, y))
self.grab_joint = pymunk.PivotJoint(
    self.cursor_body, body, (0, 0), local_anchor,
)
self.grab_joint.max_force = 50000
self.space.add(self.grab_joint)
```

- `(0, 0)` = el centro del cursor (anchor local del cursor body).
- `local_anchor` = el punto exacto donde el usuario hizo click, **convertido a coordenadas locales del body agarrado**.

Acá está la conversión mundo→local explícita. Por eso era importante entender la diferencia.

¿Y `max_force`?

```
self.grab_joint.max_force = 50000
```

Por defecto, un joint puede aplicar **fuerza infinita** para mantener la restricción. Eso rompe la física: arrastrás muy rápido, el body teletransporta detrás del cursor, atravesando paredes.

Con `max_force` el joint tira con esa fuerza máxima y nada más. Si arrastrás muy rápido el body "se queda atrás" y vuelve cuando frenás. La física se mantiene estable.

El número exacto depende de las masas. 50000 funciona bien para bodies de masa ~1; subir si los bodies son pesados.

Soltar

```
def on_mouse_release(self, x, y, button, modifiers):  
    if button != arcade.MOUSE_BUTTON_LEFT or self.grab_joint is None:  
        return  
    self.space.remove(self.grab_joint)  
    self.grab_joint = None
```

Tres líneas. Sacamos el joint del space → la restricción deja de existir → el body vuelve a caer por gravedad.

crear joint = "agarrar" • destruir joint = "soltar". Una abstracción muy potente.

Demo: 05_mouse_drag.py

```
cd 5._physics && uv run 05_mouse_drag.py
```

Cosas para mirar:

- Click sostenido sobre una caja → la podés mover libremente.
- Si arrastrás contra una pared, **la pared empuja** (max_force respeta colisiones).
- La línea verde dibuja el joint mientras existe — pymunk no lo dibuja, lo hacemos a mano leyendo `joint.anchor_b` y `local_to_world`.
- Soltar en el aire → cae.
- Click sobre el fondo → no agarra nada (`point_query_nearest` devuelve `None`).

3. Callbacks de colisión

`06_collision_callback.py`

El problema

Hasta ahora la física era una **caja negra**: los bodies chocan, rebotan, y nosotros vemos el resultado en pantalla.

Pero un juego necesita **reaccionar** a colisiones:

- Sumar puntos cuando la pelota entra al arco.
- Reproducir un sonido al chocar.
- Romper un body cuando recibe demasiado impacto.
- Activar un trampolín cuando el jugador lo pisa.

Necesitamos que pymunk nos AVISE cuando dos shapes específicas se tocan.

El patrón en 3 pasos

```
# 1. marcar las shapes con un "tipo de colision" (numero arbitrario)
BALL_TYPE = 1
GOAL_TYPE = 2
ball_shape.collision_type = BALL_TYPE
goal_shape.collision_type = GOAL_TYPE

# 2. pedir un handler para ese par de tipos
handler = self.space.add_collision_handler(BALL_TYPE, GOAL_TYPE)

# 3. asignar la funcion que pymunk va a llamar
handler.begin = self.on_goal
```

El número del `collision_type` no significa nada por sí mismo. Es solo un identificador para que el handler sepa qué shapes le interesan.

El callback

```
def on_goal(self, arbiter, space, data):  
    self.score += 1  
    self.needs_respawn = True  
    return True    # True = aceptar la colision normalmente
```

- `arbiter` = info de la colisión (las dos shapes, normal, impulso, etc.).
- `return True` → pymunk procesa la colisión como siempre.
- `return False` → pymunk la **ignora** (los bodies se atraviesan). Útil para sensores condicionales.

begin vs otros eventos

Un handler tiene **cuatro** posibles callbacks:

Callback	Cuándo
begin	una vez, al iniciar el contacto
pre_solve	cada frame mientras se tocan, antes de resolver
post_solve	cada frame mientras se tocan, después de resolver
separate	una vez, al separarse

Para "ganaste un punto" → **begin**. Para "vibra mientras toca" → **post_solve**. Para "sale humo al separarse" → **separate**.

Sensores: detectar sin empujar

```
goal_shape.sensor = True
```

Una shape con `sensor=True` **dispara los callbacks** pero **no aplica fuerzas**. La pelota la atraviesa como si no estuviera, pero el handler igual se llama.

Perfecto para zonas de gol, checkpoints, triggers, áreas de daño.

Sin `sensor=True` la pelota rebotaría contra el segment del gol y nunca lo cruzaría — el handler dispararía pero el efecto visual sería raro.

Trampa: no mutar bodies dentro del callback

```
def on_goal(self, arbiter, space, data):
    self.score += 1
    # NO: self.space.remove(self.ball_body, self.ball_shape)
    self.needs_respawn = True    # <- agendar para despues
    return True
```

Pymunk está **a la mitad** de iterar contactos cuando llama al callback. Si removés/agregás bodies ahí adentro, podés invalidar el iterador y crashear (o peor: silently corromper el estado).

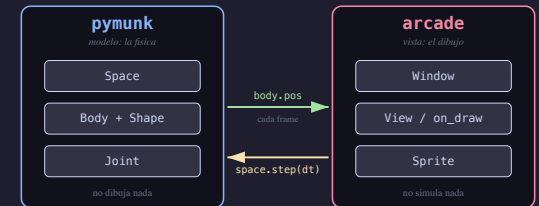
```
def on_update(self, delta_time):
    self.space.step(1/60)
    if self.needs_respawn:          # procesar despues del step
        self.spawn_ball()
        self.needs_respawn = False
```

La escena: pachinko

- Pelota cae rebotando entre clavos (`Circle` estáticos).
- Línea inferior con `collision_type = GOAL_TYPE` y `sensor=True`.
- Callback `BALL_TYPE x GOAL_TYPE` → sumar gol + respawn.

Es el ejemplo mínimo que junta **todo**:

- bodies estáticos (clavos)
- body dinámico (pelota)
- `collision_type`
- `sensor`
- handler con `begin`



Demo: 06_collision_callback.py

```
uv run 06_collision_callback.py
```

Mirar:

- La pelota rebota entre clavos (elasticidad alta).
- Al cruzar la línea verde → score sube y respawnea arriba.
- El respawn es en x **aleatoria** → cada pelota toma un camino distinto entre los clavos (caos determinista).

Para extender:

- Bajar `elasticity` de los clavos a `0.2` → caída mucho más rápida.
- Sacar `sensor=True` del gol → la pelota rebota en la línea, no la cruza.
- Cambiar `gravity` a `(50, -500)` → todas las pelotas terminan en el mismo lado.

Cierre

Lo nuevo en esta clase

Concepto	Para qué sirve
Anchors locales en joints	armar estructuras correctamente
Body KINEMATIC	controlar bodies "a mano" desde input
Joints dinámicos	crear/destruir restricciones en runtime
<code>point_query_nearest</code>	preguntarle al space "¿qué hay acá?"
<code>world_to_local</code>	convertir clicks a anchors
<code>max_force</code> en joints	drag estable sin teletransporte
<code>collision_type</code> + handler	que pymunk nos avise de colisiones
<code>sensor=True</code>	detectar contacto sin empujar
Mutación diferida	nunca cambiar el space adentro de un callback

El patrón juntando todo

```
class GameView(arcade.View):
    def __init__(self):
        self.space = pymunk.Space()
        # ... bodies, shapes, collision_types ...
        handler = self.space.add_collision_handler(A, B)
        handler.begin = self.on_hit

    def on_mouse_press(self, x, y, button, mods):
        # crear/quitar joints contra un cursor KINEMATIC

    def on_hit(self, arbiter, space, data):
        self.pending_changes.append(...) # diferido
        return True

    def on_update(self, delta_time):
        self.space.step(1/60)
        for change in self.pending_changes: change()
        # ... sync body -> sprite ...
```

Resumen del módulo entero

Clase 1 (06_physics):

- Space · Body · Shape
- body ↔ sprite
- Joints estáticos
- Composición (auto)

Clase 2 (esta):

- Anchors locales bien
- Input → física (mouse)
- Eventos ← física (callbacks)
- Mutación diferida

Con esto ya podés armar un juego completo: bodies con sus formas, gravedad, joints, input del usuario y reglas de juego disparadas por colisiones.

¿Preguntas?

Tarea: combinar los tres ejemplos en uno solo — un mini-pinball donde el usuario puede mover los flippers con el mouse (`mouse_drag`), la bola dispara un score al pasar por sensores (`collision_callback`), y todo armado con joints correctamente anclados.

Próxima clase: transformaciones 2D/3D (módulo 6) — hacer a mano las matemáticas que pymunk hace por nosotros.